

NBA Shot Make Probability

Theodore Jun | Analyst Intern Project

Objective and approach

I built a model that estimates make probability for each supplied non-fouled NBA shot. My starting idea was to combine the difficulty of the attempt with the shooter's ability from that area of the floor. The final submission uses XGBoost with shot location, a shrunk shooter-zone percentage, shot type, defensive pressure, shooter movement, and game context. Boosted trees learn nonlinear relationships and interactions between these features. Only the supplied datasets were used.

How the features describe a shot

Feature group	Final inputs and purpose
Location	13 custom zones, distance from the rim, x and y coordinates, and absolute y distance from the centerline. Zones provide broad areas; coordinates preserve variation within them.
Shooter ability	Shooter-zone make probability shrunk toward the corresponding zone average, with $k = 200$. Shooter identity is used to build this estimate, not entered as a raw model category.
Shot type and movement	Seven shot-type categories; shooter speed at release; dribbles before the shot; and a no-dribbles flag. Zero dribbles does not necessarily mean catch-and-shoot.
Defensive pressure	Closest-defender distance; four pre-release distances; two closing rates; contest-count category; and $\log(1 + \text{nearest contesteer distance})$.
Game context	Shot-clock time and the halfcourt, transition, or direct-second-chance category.

Location and defensive definitions

The rim is at $(-41.75, 0)$. My zones separate the 5-foot rim area, the remaining paint, the free-throw-line area, left/right midrange, corners and wings, top-of-key threes, deep threes, half-court attempts, and outside-court coordinates. Deep threes are at least 3 feet beyond the nearest part of the three-point line. The recorded three-point indicator distinguishes twos from threes. Unusual outside-court records remain available for prediction.

Closing rates equal earlier separation minus release separation, divided by elapsed time, over one second and the final quarter-second. They reflect relative movement by both players. Contest counts use None, One, Two, and Three or more. For uncontested shots, the distance term is zero and the None category identifies why; no shot is dropped simply because there is no contesteer.

Stabilizing shooter percentages

Shooter-zone probability = $(\text{makes} + 200 \times \text{zone probability}) / (\text{attempts} + 200)$. A player with 2 makes in 2 attempts in a 35% zone receives an estimate of 35.64%, rather than 100%. As attempts increase, the player's own results carry more weight. An unseen shooter-zone combination falls back to the zone average.

Validation and model behavior

Keeping outcomes out of their own predictors

The main split fits on 212,458 shots and validates on 213,261 shots from season fe055. Five random folds within the fitting data generate shooter statistics: both the zone average and player-zone results for a shot come from the other four folds. Validation features use fitting data only. I also reversed the season roles to check whether improvements held in the other direction; the anonymized IDs do not establish chronological order.

Observed-outcome log loss

Lower log loss is better. The table compares development results against actual makes and misses. The assignment evaluates against an internal model's probabilities, so these local scores are a proxy for the final assessment.

Model	Original split	Reverse split
Zone average	0.6601794	0.6637450
Shooter + zone, k = 200	0.6591635	0.6626906
Curved regression/GAM + contest information	0.6309523	0.6354833
Initial XGBoost	0.6274871	0.6314379
XGBoost + precise location	0.6254356	0.6292999
Final: location + full defender approach	0.6252115	0.6289117

Precise location improved both directions by about 0.0021 log loss. Retaining the four defender distances in addition to closing rates added a smaller improvement in both directions. The final original-split loss is about 5.3% lower than the zone-only baseline; this is a relative reduction in log loss, not a percentage-point increase in shooting accuracy.

What the final model uses most

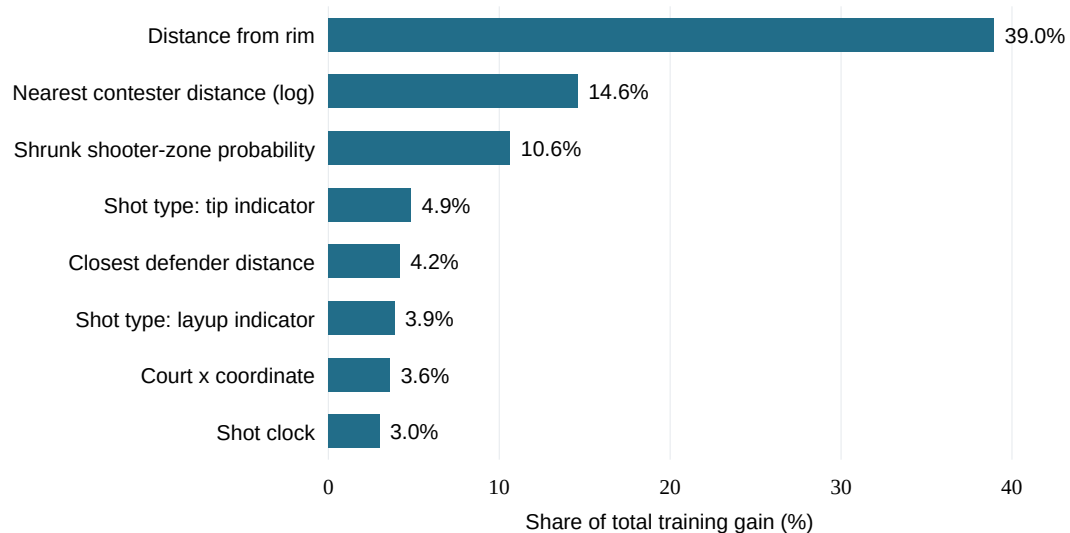


Figure 1. Top eight encoded features by normalized XGBoost gain from the full-data refit. Gain measures training-objective improvement from splits using a feature. Shot-type indicators are separate columns, so this chart does not aggregate all shot-type importance. Correlated inputs can share importance; these values are not causal effects or held-out ablation results.

Lessons, limitations, and final fit

What I learned from the experiments

The initial zone averages provided an interpretable baseline. Shrinkage addressed the small-sample problem in individual shooting percentages. Logistic regression, shot-type interactions, and curved distance effects improved that baseline before boosted trees produced further gains. I checked calibration by comparing average predicted and observed make rates within probability groups and shot types. Large gaps in small groups are not automatically the best opportunity to reduce overall loss; both sample size and error size matter.

A major issue was shooter-history coverage: about 21.1% of validation shots came from shooters absent from the fitting season, while almost all out-of-fold fitting shots retained some player history. A hidden-history training experiment tried to represent that situation, but validation loss changed from 0.6274871 to 0.6275443, slightly worse. I did not retain that change. The result illustrates why a plausible explanation for an error still needs an empirical test.

A separate broader comparison explored other model families and combinations. A selected four-model average reached approximately 0.6237085 on the original split and 0.6273382 in reverse. Those experiments remain separate: the submitted predictions use the original XGBoost workflow documented here, not that ensemble. The broader results were also obtained during development, not on an untouched final test.

Limitations and possible next steps

Both season directions were examined repeatedly while selecting features and settings. They are development checks, not unbiased estimates from an untouched test set. With more data, I would reserve a later season for a single final evaluation and tune on earlier seasons. The provided test outcomes are unavailable, and I cannot report a test-set score.

Sparse or missing player history remains a limitation. Random shot-level folds also give a different amount of shooter history from a season-level holdout. Future work could test player-grouped or time-aware feature construction and more structured pooling across related shot areas. Additional seasons could help distinguish stable shooter ability from sampling variation.

The features describe association, not causation. Nearest-defender measurements do not identify contest direction, shooter speed does not fully describe shot preparation, and coordinate anomalies may contain measurement error. The model applies to the supplied non-fouled shot population. I would investigate these data-quality and coverage issues before adding many more model parameters.

Reproducible final refit and submission

The final model uses a binary logistic objective with histogram trees, learning rate 0.03, maximum depth 4, minimum child weight 20, row and column sampling of 0.8, L2 penalty 5, and seed 2027. Early stopping on the original split selected 1,163 rounds; the reverse check uses that fixed count. The final refit also uses 1,163 rounds on all 425,719 labeled shots, with fresh five-fold shooter features. Test shooter statistics come from all labeled training data.

The script reads the compressed CSVs directly and writes 213,977 make probabilities. Checks confirm finite values between 0 and 1, the required `shot_id` and `make_prob` columns, and unchanged template row order. The final refit's training performance is not reported as an estimate of generalization.

AI assistance

AI tools assisted with R implementation, explanations, debugging, model comparisons, validation diagnostics, code cleanup, and drafting this write-up. The project developed iteratively around my proposed shot zones, shooter-ability estimates, and predictor choices. The accompanying prompt log records the assistance used.